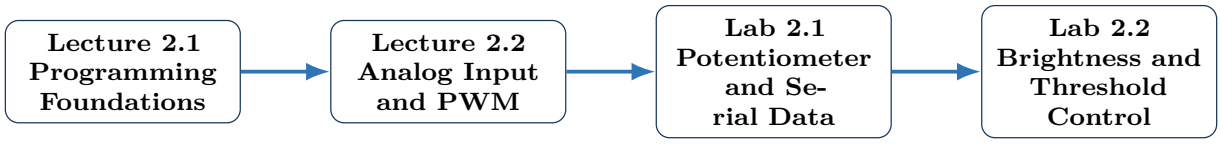
Week 2: Arduino Programming and Analog Control

Lecture 2.1 and Lecture 2.2

ENGR 1105 | Fall 2026



Week 2 Goal

By the end of this week, you should be able to read a short Arduino program, explain how information moves through the program, inspect measured values in the Serial Monitor, and use an analog input to control an output. These skills prepare you for Week 3, when the potentiometer is replaced by physical sensors.

Meeting	Main focus
Lecture 2.1	Variables, constants, program flow, conditionals, Boolean logic, and Serial Monitor
Lecture 2.2	Analog signals, potentiometers, `analogRead()`, voltage conversion, PWM, and `map()`
Lab 2.1	Read a potentiometer and inspect the measurements in the Serial Monitor
Lab 2.2	Control LED brightness and create threshold-based behavior

Opening Questions

1. What information must a program remember while it is running?
2. Why does a potentiometer produce more than two possible input values?
3. How can a digital Arduino pin make an LED appear partly bright?

Lecture 2.1

Arduino Programming Foundations

Monday, November 2, 2026

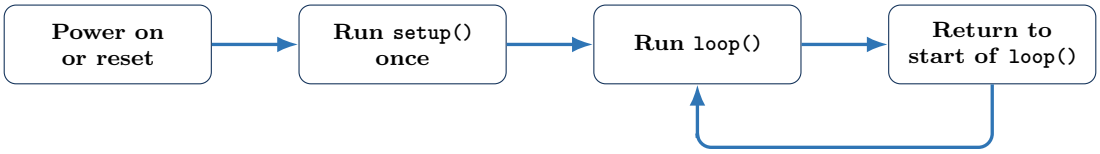
Learning Objectives

By the end of Lecture 2.1, you should be able to:

1. explain the roles of `setup()` and `loop()`;
2. create and use variables and constants;
3. trace the order in which program statements run;
4. use comparison and Boolean expressions in an `if` statement;
5. send values to the Serial Monitor; and
6. use a controlled debugging process.

1. A Program Is a Sequence of Instructions

An Arduino program is called a **sketch**. The sketch tells the microcontroller what hardware is connected, what information to read, what decisions to make, and what output commands to send.



`setup()`

The function `setup()` runs one time when the Arduino starts or resets. It is normally used to configure pins and start communication tools such as the Serial Monitor.

`loop()`

The function `loop()` runs repeatedly. It normally reads inputs, performs calculations or decisions, sends output commands, and then repeats.

Example: Repeated Control

A button-controlled light repeatedly performs the following sequence:

1. read the button;
2. decide whether the button is pressed;

3. set the LED output;
4. return to the first step.

The loop must repeat because the user may press or release the button at any time.

2. Variables and Constants

A program needs names for the values it uses. Some values stay fixed, while other values change while the program runs.

Constant

A **constant** is a named value that should not change while the program runs. Pin numbers are commonly stored as constants.

```
const int buttonPin = 2;
const int ledPin = 8;
```

Variable

A **variable** is a named storage location whose value may change while the program runs.

```
int buttonState = HIGH;
int sensorValue = 0;
```

Data type	Typical use	Example
`int`	Whole numbers	sensor reading, pin number, delay time
`float`	Numbers with decimals	calculated voltage or physical measurement
`bool`	Logical state	`true` or `false`
`unsigned long`	Large nonnegative time values	elapsed time from `millis()`

Choose a Meaningful Name

A name such as `sensorValue` is easier to understand than a name such as `x`. Good names make the program easier to read, explain, and debug.

3. Assignments and Program State

The assignment operator `=` stores a value in a variable.

```
sensorValue = analogRead(A0);
```

This statement means:

1. read the electrical value at analog pin A0;
2. convert the measurement to a number;
3. store the number in `sensorValue`.

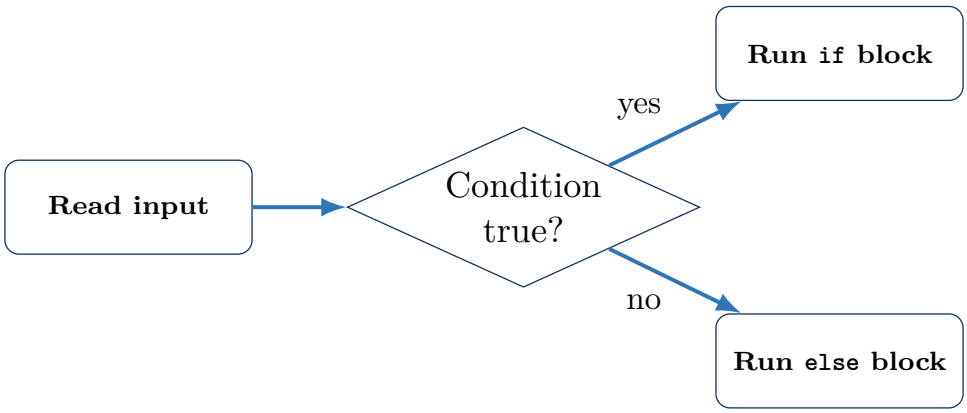
Do Not Confuse = and ==

- `=` assigns a value: `buttonState = digitalRead(2);`
- `==` compares two values: `buttonState == LOW`

4. Conditions and Decisions

An `if` statement runs a section of code only when a condition is true.

```
if (buttonState == LOW) {
  digitalWrite(ledPin, HIGH);
} else {
  digitalWrite(ledPin, LOW);
}
```



Comparison Operators

Operator	Meaning
`==`	equal to
`!=`	not equal to
`>`	greater than
`<`	less than
`>=`	greater than or equal to
`<=`	less than or equal to

Boolean Operators

More than one condition can be combined.

Operator	Name	Meaning
`&&`	AND	Both conditions must be true.
`||`	OR	At least one condition must be true.
`!`	NOT	Reverses true and false.

Example: Safe Operating Range

Suppose a system should turn on only when a value is greater than 300 and less than 700.

```
if (sensorValue > 300 && sensorValue < 700) {
  digitalWrite(ledPin, HIGH);
} else {
  digitalWrite(ledPin, LOW);
}
```

5. Program Tracing

Program tracing means following the statements in order and recording how each variable changes.

```
int value = 200;
value = value + 100;

if (value >= 300) {
  digitalWrite(ledPin, HIGH);
}
```

Trace the Program

1. `value` begins at 200.
2. `value = value + 100;` changes it to 300.
3. The condition `value >= 300` is true.
4. The LED output is set HIGH.

Class Practice

Trace this code for `sensorValue = 650`:

```
if (sensorValue < 300) {
  level = 0;
} else if (sensorValue < 700) {
  level = 1;
} else {
  level = 2;
}
```

What value is stored in `level`? Explain which condition is checked first and which block runs.

6. Communicating with the Serial Monitor

The Serial Monitor displays text and numbers sent from the Arduino to the computer. It is useful for observing values that cannot be seen directly.

```
void setup() {
  Serial.begin(9600);
}

void loop() {
  int sensorValue = analogRead(A0);
  Serial.println(sensorValue);
  delay(100);
}
```

Serial Communication Sequence

1. `Serial.begin(9600)` starts communication.
2. `Serial.print()` sends data without automatically moving to a new line.
3. `Serial.println()` sends data and then starts a new line.
4. The baud rate in the Serial Monitor must match the value in the program.

> **Do Not Use Pins 0 and 1 in the Starter Labs**
>
> On an Arduino Uno, digital pins 0 and 1 are used for serial communication. Connecting other components to these pins can interfere with program upload and Serial Monitor operation.

7. Timing and `delay()`

The function `delay()` pauses the program for a chosen number of milliseconds.

```
digitalWrite(ledPin, HIGH);
delay(500);
digitalWrite(ledPin, LOW);
delay(500);
```

During the delay, the program is not reading new input values. This is acceptable for simple starter programs, but long delays can make an interactive system feel unresponsive.

Preview: Nonblocking Timing

Later in the course, `millis()` can be used to measure elapsed time without stopping the entire program. For Week 2, use short delays when repeated measurements are required.

8. A Controlled Debugging Process

When a program does not behave as expected, do not change many things at once.

1. Read the first compiler error.
2. Confirm the board and port.
3. Test the smallest working version.
4. Print important variable values to the Serial Monitor.
5. Change one statement or one wire.
6. Test again and record what changed.

Observed problem	First checks
Program does not compile	Missing semicolon, unmatched brace, misspelled command, capitalization
Program uploads but output is wrong	Pin number, pin mode, condition, variable value
Serial Monitor is blank	Correct baud rate, correct port, `Serial.begin()`
Values never change	Wiring, analog pin, component orientation, assignment statement

Lecture 2.1 Exit Questions

1. What is the difference between a constant and a variable?
2. What is the difference between `=` and `==`?
3. Why is the Serial Monitor useful during troubleshooting?
4. What is one limitation of using a long `delay()`?

Lecture 2.2

Analog Input, Potentiometers, and PWM

Tuesday, November 3, 2026

Learning Objectives

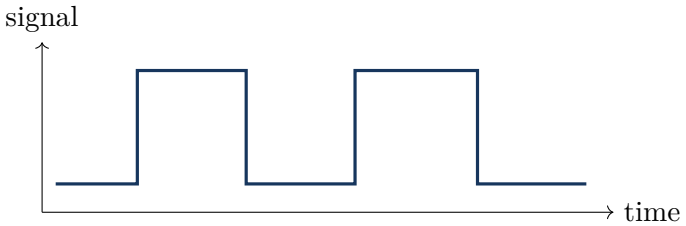
By the end of Lecture 2.2, you should be able to:

1. distinguish digital and analog signals;
2. explain how a potentiometer produces a variable voltage;
3. interpret the output of `analogRead()`;
4. convert an analog reading to an estimated voltage;
5. explain PWM and duty cycle;
6. use `map()` to connect an input range to an output range; and
7. design threshold-based behavior from analog measurements.

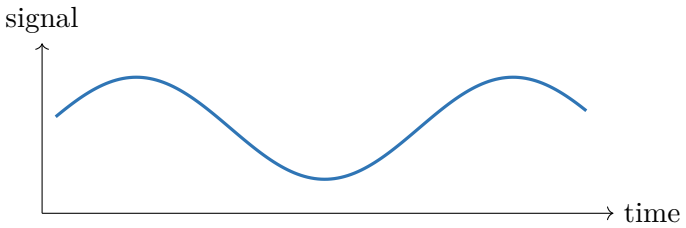
9. Digital and Analog Signals

A digital signal has two interpreted states. An analog signal can vary continuously over a range.

Digital	Analog
Two states: LOW or HIGH	Many possible voltage levels
Button, limit switch	Potentiometer, light sensor
Read with `digitalRead()`	Read with `analogRead()`
Typical result: 0 or 1	Arduino Uno result: 0 through 1023



Digital signal



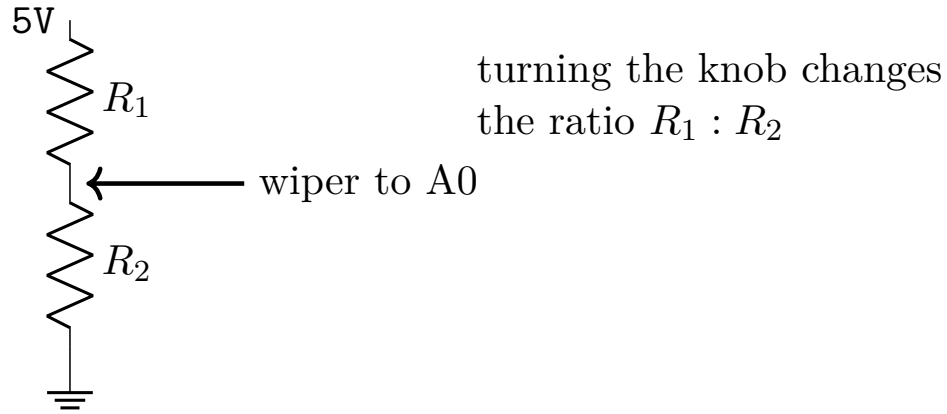
Analog signal

The Arduino Stores a Number

The analog input voltage is continuous, but the Arduino converts it to a whole-number reading. The reading is a digital representation of the measured voltage.

10. How a Potentiometer Works

A potentiometer is a three-terminal variable resistor. When connected between `5V` and `GND`, the center terminal provides an adjustable voltage.



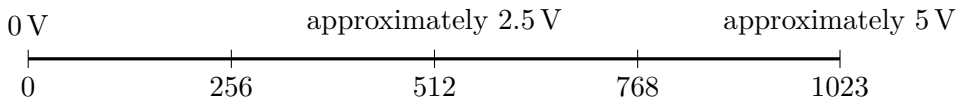
Potentiometer Connections

- one outside terminal to `5V`;
- the other outside terminal to `GND`;
- center terminal, called the wiper, to analog pin A0.

Reversing the two outside terminals reverses the direction in which the reading increases, but it does not damage the potentiometer.

11. Reading an Analog Input

On an Arduino Uno, `analogRead(A0)` returns an integer from 0 to 1023.




```
const int potPin = A0;
int sensorValue = 0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  sensorValue = analogRead(potPin);
  Serial.println(sensorValue);
  delay(100);
}
```

Analog Pins

The analog input pins are labeled A0, A1, A2, and so on. In this course, write the label directly, such as `A0`, rather than replacing it with an unexplained number.

12. Converting a Reading to Voltage

For a nominal 5 V reference, the estimated input voltage is

$$V_{\text{in}} = \frac{\text{analog reading}}{1023}\,(5\,\text{V})\,.$$

Example: Reading 614

Suppose the Serial Monitor shows 614.

$$V_{\text{in}} = \frac{614}{1023}\,(5\,\text{V}) \approx 3.00\,\text{V}.$$

The result is approximate because the board supply and reference voltage may not be exactly 5.00 V.

```
int sensorValue = analogRead(A0);
float voltage = sensorValue * (5.0 / 1023.0);

Serial.print("Reading: ");
Serial.print(sensorValue);
Serial.print("   Voltage: ");
Serial.println(voltage);
```

Why Use `5.0` and `1023.0`?

Writing decimal values tells the program to perform floating-point division. This allows the result to include a fractional voltage instead of only a whole number.

13. Real-Life Analog Inputs

Device	Analog information	Possible output
Dashboard dimmer	knob position	display brightness
Joystick	horizontal or vertical position	robot speed or direction
Thermostat	temperature measurement	heater or fan command
Light-sensitive lamp	room brightness	lamp brightness or ON/OFF state

Class Question

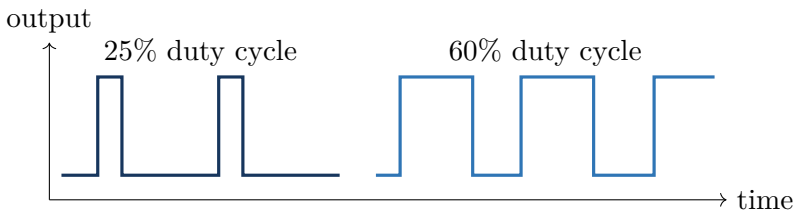
A push button and a potentiometer are both user inputs. Why is the button normally read with `digitalRead()`, while the potentiometer is normally read with `analogRead()`?

14. PWM Output

The Arduino Uno cannot create a continuously adjustable voltage from a standard digital output pin. Instead, selected pins use **pulse-width modulation**, or PWM.

Pulse-Width Modulation

PWM switches an output rapidly between HIGH and LOW. Changing the fraction of time spent HIGH changes the average energy delivered to the load.



PWM Range

`analogWrite(pin, value)` uses a value from 0 to 255.

- 0 means always LOW;
- 255 means always HIGH;
- values between 0 and 255 produce intermediate duty cycles.

On the Arduino Uno, use a PWM-capable pin marked with a tilde, such as `~9`.

15. Controlling LED Brightness

```
const int potPin = A0;
const int ledPin = 9;

void setup() {
  pinMode(ledPin, OUTPUT);
}

void loop() {
  int sensorValue = analogRead(potPin);
  int brightness = map(sensorValue, 0, 1023, 0, 255);
  analogWrite(ledPin, brightness);
}
```

`map()`

The function `map()` converts a number from one range to a corresponding number in another range.

```
map(value, inputLow, inputHigh, outputLow, outputHigh)
```

For the potentiometer example:

$$0 \ldots 1023 \quad \longrightarrow \quad 0 \ldots 255.$$

Example: Midrange Input

For an analog reading near 512,

$$\text{brightness} \approx 128.$$

The PWM duty cycle is approximately one-half, so the LED appears near half brightness.

PWM Is Not a True Analog Voltage

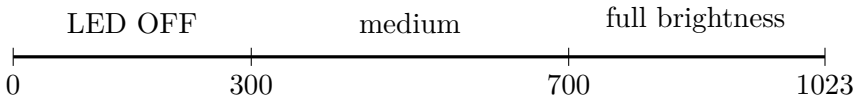
The output still switches between LOW and HIGH. The LED appears dimmer because it receives energy during only part of each rapid switching cycle.

16. Threshold-Based Behavior

Sometimes an analog value should be converted to a small number of operating states.

```
int sensorValue = analogRead(A0);

if (sensorValue < 300) {
  analogWrite(ledPin, 0);
} else if (sensorValue < 700) {
  analogWrite(ledPin, 128);
} else {
  analogWrite(ledPin, 255);
}
```



Choose Better Thresholds

Suppose the potentiometer readings in an actual circuit range only from 35 to 990. How might you choose thresholds that divide the usable range into three approximately equal regions?

17. Calibration and Measurement Variation

Real measurements are not perfectly constant. Even when the knob appears stationary, the displayed reading may change by one or two counts.

Possible causes include:

- electrical noise;
- small movement of the knob or jumper wires;
- variation in the supply voltage;
- limited analog-to-digital resolution.

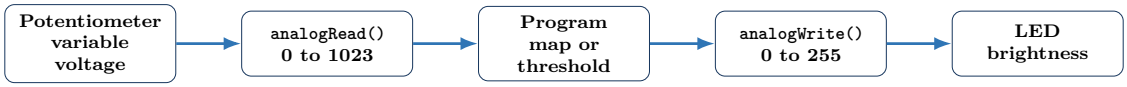
Calibration

Calibration means comparing the raw measurement with known conditions and selecting useful minimum, maximum, or threshold values for the actual system.

A simple calibration procedure is:

1. turn the potentiometer fully in one direction and record the minimum;
2. turn it fully in the other direction and record the maximum;
3. select thresholds or a mapping range based on the observed values;
4. test the full operating range again.

18. Complete Week 2 System



Sense–Decide–Act

- **Sense:** read the potentiometer voltage.
- **Decide:** map the value or compare it with thresholds.
- **Act:** command the LED brightness using PWM.

19. Week 2 Summary

Core Ideas

- `setup()` runs once; `loop()` repeats.
- Constants store fixed values; variables store values that may change.
- `=` assigns a value; `==` compares values.
- Conditions and Boolean operators allow the program to choose among behaviors.
- The Serial Monitor helps reveal measurements and program state.
- `analogRead()` converts an input voltage to a value from 0 through 1023.
- A potentiometer produces a variable voltage at its center terminal.
- PWM controls average output energy by changing duty cycle.
- `map()` connects one numerical range to another.
- Calibration uses observed measurements to choose useful ranges and thresholds.

Lecture 2.2 Exit Questions

1. What reading should A0 produce at approximately 2.5 V?
2. Estimate the voltage represented by an analog reading of 800.
3. Why does `analogWrite()` use values from 0 to 255 instead of 0 to 1023?
4. What is PWM duty cycle?
5. How does the Week 2 system demonstrate sense–decide–act?

Before Lab 2.1

Bring the Arduino Uno, USB data cable, breadboard, jumper wires, potentiometer, LED, and 330 Ω resistor. Lab 2.1 will focus on analog measurements and the Serial Monitor. Lab 2.2 will use those measurements to control LED brightness and threshold behavior.